Reporte

Nombre: Ximena Montserrat Ovalle Vallejo Santiago Almanza Moreno Javier Hernández Huerta	**Matrícula**: 1999238 2001903 2089204
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Módulo Netacad:	**Actividad**: Actividad Unidad 4. Implementación del núcleo de negocio TechStore
Fecha: 28/ 10/ 2026	

Bibliografia (Formato APA)

- Akşahin, F. (2023, 6 de septiembre). *Specification design pattern*. Medium. https://medium.com/@ferid.aksahin98/specification-design-pattern-210b5df70252
- Fowler, M. (2005, 12 de diciembre). *Domain event*. martinfowler.com. https://martinfowler.com/eaaDev/DomainEvent.html
- Jovanović, M. (2023, 23 de diciembre). *Value objects in .NET (DDD fundamentals)*. Milan Jovanović. https://milanjovanovic.tech/blog/value-objects-in-dotnet-ddd-fundamentals
- MathWorks. (s.f.). *Introducción a state machines*. Recuperado el 21 de octubre de 2026, de https://la.mathworks.com/discovery/state-machine.html
- Ramírez, S. (s.f.). *FastAPI*. FastAPI. Recuperado el 21 de octubre de 2026, de https://fastapi.tiangolo.com/#run-it

Reporte

Objetivo:

El objetivo de esta actividad fue implementar el núcleo de negocio de una tienda ficticia, TechStore, aplicando los conceptos de la Unidad 4: entidades y Value Objects con invariantes, reglas de negocio mediante el Specification Pattern con composición AND, OR y NOT, una máquina de estados que controla el ciclo de vida de los pedidos y registra Domain Events, y una API REST con FastAPI que traduce las excepciones del dominio a códigos HTTP. Todo debía demostrarse con pruebas automáticas en pytest y con la ejecución de los quince escenarios de evaluación del paquete, sin modificar las reglas para que un caso pasara.

REPOSITORIO GITHUB:

https://github.com/ximenaovallev/CLN_AU4_Equipo2

Procedimiento:

Se desarrolló un programa que administra los pedidos de una tienda ficticia. Lee clientes, productos y pedidos de archivos CSV, los convierte en objetos de dominio, aplica reglas de descuento con el Specification Pattern, controla el ciclo de vida de cada pedido con una máquina de estados que registra Domain Events y expone las operaciones mediante una API REST con FastAPI. Todo trabaja en memoria: al cerrar el programa, los cambios se pierden.

Comprobación	Resultado
python cargar_csv.py	16 pedidos en DRAFT; O001 = 4 unidades y 2100.00 MXN
python ejecutar_escenarios.py	15/15 escenarios E01–E15 cumplen su objetivo
python -m pytest -q	69 pruebas aprobadas, 0 fallos
Swagger /docs	Respuestas 200, 404, 409 y 422 en los casos indicados

1. Lectura de los CSV y cálculo manual

Relaciones entre archivos: pedidos.cliente_id → clientes.cliente_id; detalle_pedidos.pedido_id → pedidos.pedido_id; detalle_pedidos.sku → productos.sku. El subtotal de un renglón es cantidad × precio y el total del pedido es la suma de sus subtotales.

Pedido	Cliente	Renglones	Unidades	Total manual
O001	C001 · Ana Torres (VIP, activa)	SKU001 × 2 = 1200.00; SKU002 × 2 = 900.00	**4**	**2100.00 MXN**
O002	C002 · Luis Medina (no VIP, activo)	SKU001 × 1 = 600.00; SKU002 × 1 = 450.00	**2**	**1050.00 MXN**
O003	C003 · Carla Ruiz (VIP, inactiva)	SKU001 × 2 = 1200.00; SKU002 × 2 = 900.00	**4**	**2100.00 MXN**
O004	C004 · Diego Peña (no VIP, inactivo)	SKU004 × 1 = 850.00	**1**	**850.00 MXN**
O005	C005 · Sofia Lara (VIP, activa) · promoción	SKU001 × 2 = 1200.00; SKU002 × 2 = 900.00	**4**	**2100.00 MXN**
O006	C006 · Pedro Soto (no VIP, activo)	SKU008 × 1 = 18000.00	**1**	**18000.00 MXN**
O007	C007 · Mariana Gil (VIP, activa)	Sin renglones	**0**	**0.00 MXN**
O008	C008 · Raul Vega (no VIP, activo)	SKU003 × 20 = 2400.00	**20**	**2400.00 MXN**
O009	C009 · Elena Mora (VIP, activa)	SKU003 × 19 = 2280.00	**19**	**2280.00 MXN**
O010	C010 · Jorge Luna (no VIP, activo)	SKU003 × 1 = 120.00; SKU009 × 1 = 380.00	**2**	**500.00 MXN**
O011	C011 · Valeria Rios (VIP, activa)	SKU001 × 1 = 600.00; SKU002 × 2 = 900.00	**3**	**1500.00 MXN**
O012	C012 · Oscar Nieto (no VIP, activo)	SKU004 × 1 = 850.00	**1**	**850.00 MXN**
O013	C001 · Ana Torres (VIP, activa)	SKU001 × 1 = 600.00	**1**	**600.00 MXN**
O014	C002 · Luis Medina (no VIP, activo)	SKU001 × 1 = 600.00	**1**	**600.00 MXN**
O015	C005 · Sofia Lara (VIP, activa)	SKU002 × 1 = 450.00	**1**	**450.00 MXN**
O016	C007 · Mariana Gil (VIP, activa)	SKU011 × 1 = 2100.00	**1**	**2100.00 MXN**

Los 16 valores coinciden con la salida de python cargar_csv.py. Las filas resaltadas (O001, O006 y O010) son las que pide la actividad corta y además se comprueban automáticamente en la prueba test_tabla_calculo_manual. O007 no tiene renglones, por eso su total es 0.00; O008 alcanza exactamente el límite de 20 unidades y O009 queda en 19.

2. Modelo de dominio

Elemento	Tipo DDD	Invariantes
Money	Value Object inmutable	Usa Decimal (rechaza float), ≥ 0 y finito; add() exige la misma moneda y devuelve un objeto nuevo
OrderItem	Value Object inmutable	SKU no vacío, cantidad entera > 0; el subtotal se calcula, no se captura
Customer	Entidad (identidad id)	id no vacío; atributos is_vip y active
Order	Entidad · Aggregate Root	Máximo 20 unidades, solo MXN, artículos solo en DRAFT, transiciones controladas
DomainEvent	Objeto inmutable	Nombre en pasado, order_id, occurred_at (UTC) y details

Jerarquía de excepciones

```
DomainException
└── OrderException
      ├── InvalidOrderData          → HTTP 422
      ├── InvalidOrderTransition    → HTTP 409
      └── OrderItemLimitExceeded    → HTTP 422
NotFound (capa de aplicación)       → HTTP 404
```

La jerarquía permite capturar cualquier violación de regla con DomainException o un caso específico con su clase. La capa HTTP decide el código de respuesta; el dominio solo expresa qué regla se violó.

3. Specification Pattern

Specification	Condición
MinimumAmountSpec(Decimal("500"))	Total **estrictamente mayor** que el umbral (>, no >=)
VIPCustomerSpec()	El cliente es VIP
MinimumUnitsSpec(4)	Al menos 4 unidades (suma de cantidades)
ActivePromotionSpec()	El pedido tiene promoción activa

Reglas compuestas (AND, OR y NOT)

```
TEN_PERCENT  = (MinimumAmountSpec(Decimal("500")) & VIPCustomerSpec()
                & MinimumUnitsSpec(4) & ~ActivePromotionSpec())
FIVE_PERCENT = MinimumAmountSpec(Decimal("5000")) &
~ActivePromotionSpec()
ANY_DISCOUNT = TEN_PERCENT | FIVE_PERCENT
```

DiscountPolicy.evaluate() aplica primero el 10 % y, solo si no se cumple, el 5 %. Ninguna Specification modifica el pedido: únicamente responde verdadero o falso.

Pedido	Regla 10 %	Regla 5 %	Resultado
O001	Sí cumple	No (2100 ≤ 5000)	**10 %**: 2100.00 − 210.00 = 1890.00
O006	No (no VIP, 1 unidad)	Sí cumple	**5 %**: 18000.00 − 900.00 = 17100.00
O005	No (promoción activa)	No (promoción activa)	**0 %**
O010	No (500 no es > 500)	No	**0 %**

4. Máquina de estados

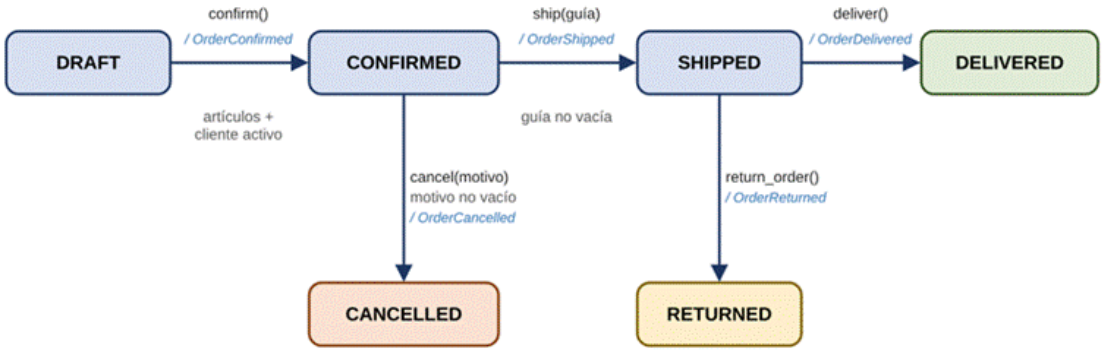


Figura 1. Ciclo de vida de Order: transiciones, condiciones y eventos emitidos.

Operación	Estado requerido	Condición adicional	Si falla
add_item(item)	DRAFT	unidades + cantidad ≤ 20; MXN	InvalidOrderTransition / OrderItemLimitExceeded / InvalidOrderData
confirm()	DRAFT	Artículos y cliente activo	InvalidOrderTransition / InvalidOrderData
ship(guía)	CONFIRMED	Guía no vacía	InvalidOrderTransition / InvalidOrderData
cancel(motivo)	CONFIRMED	Motivo no vacío	InvalidOrderTransition / InvalidOrderData
deliver()	SHIPPED	—	InvalidOrderTransition
return_order()	SHIPPED	—	InvalidOrderTransition

Ejemplos de transiciones inválidas: cancelar desde SHIPPED (E13), confirmar desde CONFIRMED (E12), cancelar desde DRAFT y entregar desde CONFIRMED. DELIVERED, CANCELLED y RETURNED son estados finales: no admiten más transiciones.

5. Domain Events

Un Domain Event registra un hecho que **ya ocurrió**; por eso su nombre está en pasado. Cada método de transición sigue siempre el mismo orden: **validar → cambiar el estado → registrar el evento**. Si una validación falla, la excepción se lanza antes de tocar el estado, así que el pedido queda igual y no se anuncia un hecho que nunca sucedió.

Evento	Se emite en	Datos registrados (details)
OrderConfirmed	confirm()	customer_id, total, units
OrderShipped	ship()	customer_id, tracking_number
OrderCancelled	cancel()	customer_id, reason
OrderDelivered	deliver()	customer_id
OrderReturned	return_order()	customer_id

Todos incluyen además order_id y occurred_at en UTC. El pedido acumula los eventos en order.events; en un sistema real, una unidad de trabajo los publicaría a suscriptores (correo, facturación, inventario) sin que Order conozca a esos suscriptores, lo que reduce el acoplamiento. Ejemplo verificado en E14: OrderConfirmed → OrderShipped → OrderReturned.

6. Bitácora de escenarios E01–E15

Cada escenario se ejecutó con un load_fixture() nuevo, de modo que los cambios de un caso no afectan al siguiente. No se editaron los pedidos para forzar resultados. Todos los pedidos inician en DRAFT.

Caso	Pedido	Operación	Objetivo	Resultado real	Evento(s)
E01	O0001	evaluar_descuento	10%	Descuento 10% → 1890.00 MXN	(ninguno)
E02	O0002	evaluar_descuento	0%	Descuento 0% → 1050.00 MXN	(ninguno)
E03	O0003	confirmar	rechazo	Rechazo (InvalidOrderData) · queda DRAFT	(ninguno)
E04	O0005	evaluar_descuento	0%	Descuento 0% → 2100.00 MXN	(ninguno)
E05	O0006	evaluar_descuento	5%	Descuento 5% → 17100.00 MXN	(ninguno)
E06	O0007	confirmar	rechazo	Rechazo (InvalidOrderData) · queda DRAFT	(ninguno)
E07	O0008	confirmar	exito	Éxito · queda CONFIRMED	OrderConfirmed
E08	O0009	agregar SKU003 x2	rechazo	Rechazo (OrderItemLimitExceeded) · queda DRAFT	(ninguno)
E09	O0010	evaluar_descuento	0%	Descuento 0% → 500.00 MXN	(ninguno)
E10	O011	evaluar_descuento	0%	Descuento 0% → 1500.00 MXN	(ninguno)
E11	O012	confirmar y enviar sin guia	rechazo envio	Rechazo (InvalidOrderData) · queda CONFIRMED	OrderConfirmed
E12	O013	confirmar dos veces	rechazo segunda	Rechazo (InvalidOrderTransition) · queda CONFIRMED	OrderConfirmed

E13	O014	confirmar, enviar y cancelar	rechazo cancelacion	Rechazo (InvalidOrderTransition) · queda SHIPPED	OrderConfirmed, OrderShipped
E14	O015	confirmar, enviar y devolver	exito	Éxito · queda RETURNED	OrderConfirmed, OrderShipped, OrderReturned
E15	O016	confirmar, enviar y entregar	exito	Éxito · queda DELIVERED	OrderConfirmed, OrderShipped, OrderDelivered

Resultado: **15 de 15** escenarios cumplen su objetivo. En todos los rechazos (E03, E06, E08, E11, E12 y E13) se verificó que el estado, las unidades y la lista de eventos quedaron sin cambios.

Explicación por escenario

Caso	Precondición clave y explicación
E01	AND VIP + monto >500 + unidades >=4 + NOT promo
E02	No VIP, total menor a 5000
E03	Cliente inactivo
E04	Promocion activa bloquea descuento
E05	OR total >5000 y sin promocion
E06	Pedido sin articulos
E07	20 unidades permitidas
E08	21 unidades excede limite
E09	Umbral estricto >500
E10	Solo 3 unidades, requiere 4
E11	La confirmacion es valida pero falta guia
E12	No permite confirmar desde CONFIRMED
E13	No permite cancelar desde SHIPPED
E14	Termina en RETURNED
E15	Termina en DELIVERED

7. API REST con FastAPI

El Controller recibe la petición, llama a OrderService y devuelve un DTO. No contiene reglas de negocio: el dominio decide si la operación es válida y la API traduce las excepciones a códigos HTTP.

Método	Ruta	Descripción	Códigos
GET	/orders/{id}	Detalle del pedido	200, 404
GET	/orders/{id}/discount	Evalúa el descuento	200, 404
POST	/orders	Crea un pedido DRAFT	201, 404
POST	/orders/{id}/items	Agrega un artículo	200, 404, 409, 422
POST	/orders/{id}/confirm	Confirma	200, 404, 409, 422
POST	/orders/{id}/ship	Envía con guía	200, 404, 409, 422
POST	/orders/{id}/cancel	Cancela con motivo	200, 404, 409, 422
POST	/orders/{id}/deliver	Entrega	200, 404, 409
POST	/orders/{id}/return	Devuelve	200, 404, 409

Prueba en Swagger	Código	Motivo
Confirmar O013 (primera vez)	200	Pedido DRAFT con artículos y cliente activo
Confirmar O013 (segunda vez)	409	Solo DRAFT puede confirmarse
Confirmar O007	422	Pedido sin artículos
Confirmar O999	404	El pedido no existe

8. Reflexión: errores encontrados

1. **ModuleNotFoundError: No module named 'app.domain'.** Al ejecutar python cargar_csv.py antes de escribir el dominio, el cargador falló porque importa Customer, Order, OrderItem y Money. Lo solucioné creando app/domain.py con esos nombres y con el constructor Order(customer, id, has_promotion) que espera el cargador. Aprendí que el cargador es un contrato: mis clases deben respetar sus nombres y parámetros.
2. **El pedido vacío O007 mostraba total 0 en lugar de 0.00.** La suma empezaba en Decimal("0"), que no tiene decimales. Lo corregí iniciando en Decimal("0.00"). Aprendí que Decimal conserva la escala y que por eso es adecuado para representar dinero.
3. **El paquete traía una carpeta __pycache__.** Subirla a GitHub incumpliría la entrega. Agregué un .gitignore con .venv/, __pycache__/ y *.pyc, y no copié esa carpeta al proyecto.

Además validé las pruebas rompiendo reglas a propósito: al cambiar > por >= en MinimumAmountSpec fallaron 2 pruebas (O010 con 500 exactos) y al permitir 21 unidades fallaron 5. Con el código correcto pasan las 69. Aprendí que una prueba útil debe fallar cuando la regla se rompe y que, si una prueba falla, se corrige la implementación y no el resultado esperado.

9. Preguntas para el reporte

¿Por qué se usa Decimal en lugar de float para precios?

float es binario y no representa exactamente valores como 0.1; por ejemplo, 0.1 + 0.2 da 0.30000000000000004. Decimal representa los centavos con exactitud y conserva la escala (600.00), así que sumas y descuentos no acumulan errores. Además se construye desde texto, Decimal("600.00"), no desde un float.

¿Cuál es la diferencia entre contar renglones y sumar unidades?

Un renglón es un OrderItem y puede tener varias unidades. O008 tiene 1 renglón con 20 unidades. El límite es de 20 unidades, por eso units suma las cantidades: O009 tiene 19 unidades y agregar 2 más (21) se rechaza aunque solo serían 2 renglones.

¿Qué ventaja tiene encapsular una regla en Specification?

La regla tiene nombre, se prueba por separado y se combina con &, | y ~ sin duplicar condiciones en la API u otros servicios. Si cambia el umbral de 500, se edita en un solo lugar, y como la Specification solo evalúa, no puede alterar el pedido.

¿Por qué los eventos se registran solo después de una transición válida?

Un evento afirma que algo ya sucedió. Si se registrara antes de validar, una operación rechazada (por ejemplo, el segundo confirm() de E12) anunciaría un hecho falso y los suscriptores podrían enviar correos o facturas indebidas.

¿Qué parte del código podrías reutilizar si reemplazas FastAPI por otra interfaz?

Todo app/domain.py, app/specifications.py y app/services.py, además de cargar_csv.py y las pruebas de dominio. Solo cambiaría app/api.py. Prueba de ello es ejecutar_escenarios.py, que usa el mismo dominio desde la consola, sin servidor web.

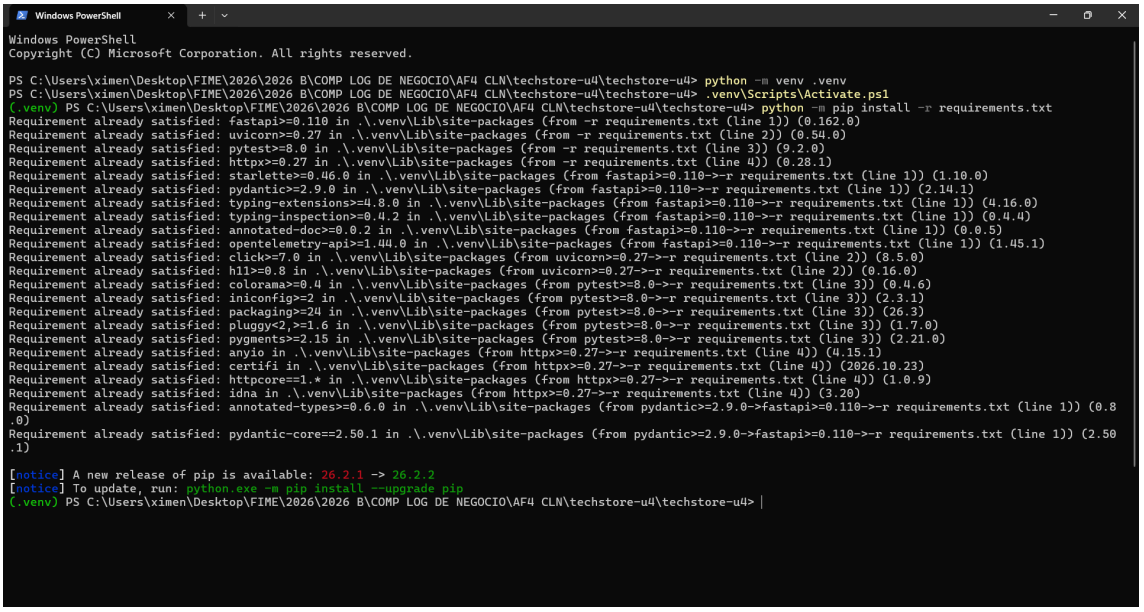
10. Lista de verificación

Punto	Estado	Evidencia
Puedo ejecutar python cargar_csv.py sin errores	Cumple	Captura 01_carga_16_pedidos
O001 muestra 4 unidades y 2100.00 MXN	Cumple	Captura 01; prueba test_pedido_o001_inicia_draft
La regla del 10 % requiere VIP, >500, ≥4 unidades y ausencia de promoción	Cumple	TEN_PERCENT; escenarios E01, E04, E09, E10
La regla del 5 % requiere >5000 y ausencia de promoción	Cumple	FIVE_PERCENT; escenario E05
Una operación inválida no cambia el estado ni añade eventos	Cumple	Bitácora; E03, E06, E08, E11, E12, E13
La API responde 404, 409 y 422 en los casos indicados	Cumple	Capturas 08, 09 y 10 de Swagger; test_api.py
pytest termina sin fallos	Cumple	Captura 05_pytest (69 passed)
El repositorio contiene README y evidencias	Cumple	README.md, docs/ y evidencias/

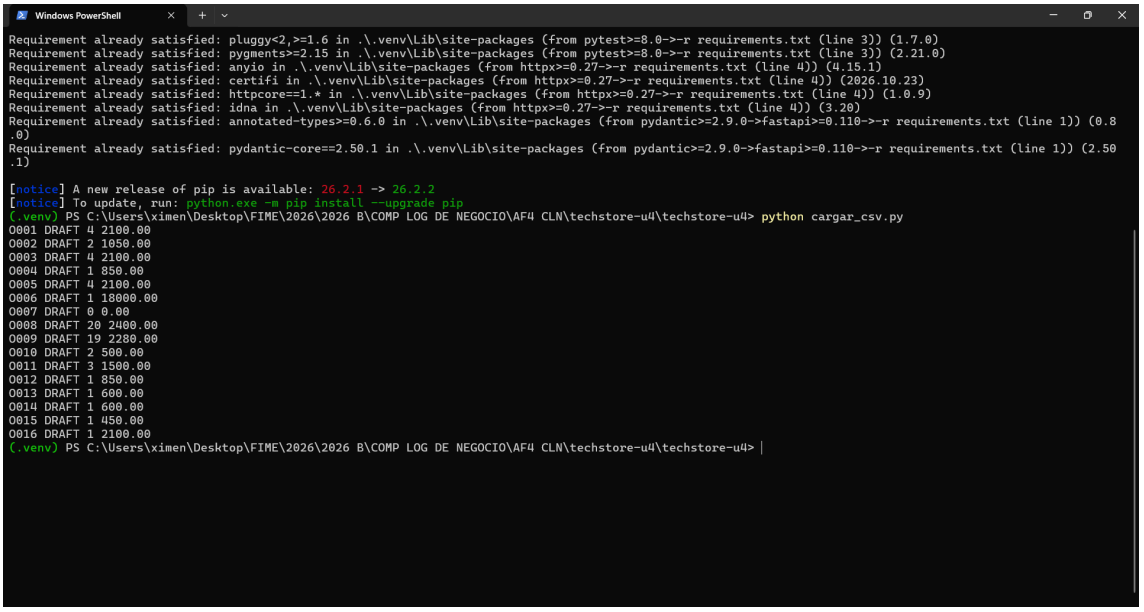
Reporte

Evidencias:

Crear el entorno e instalar las dependencias

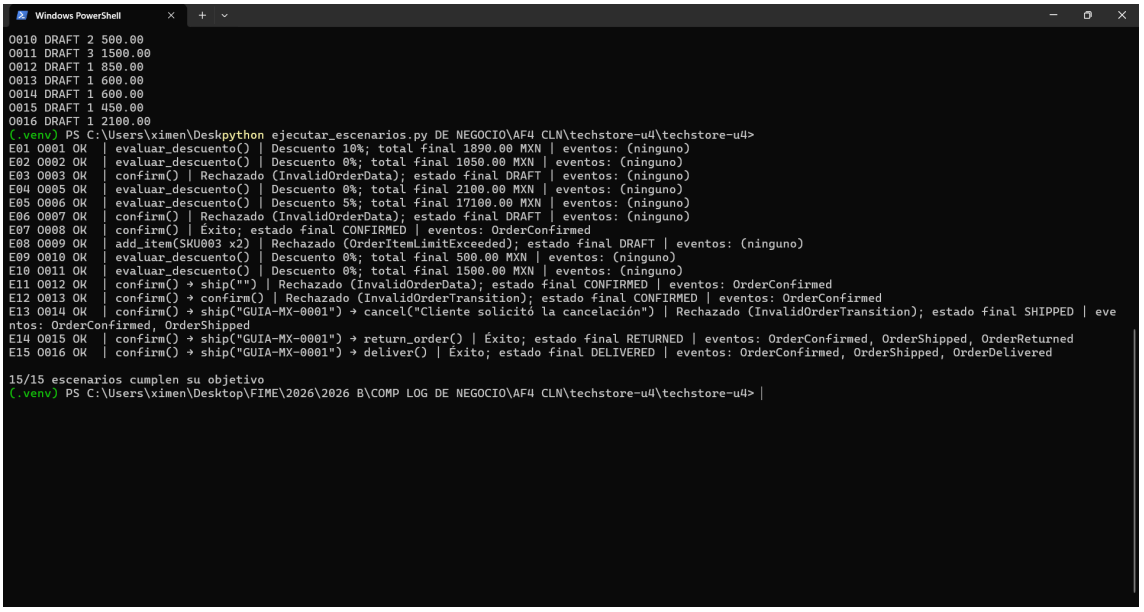


Carga de los 16 pedidos

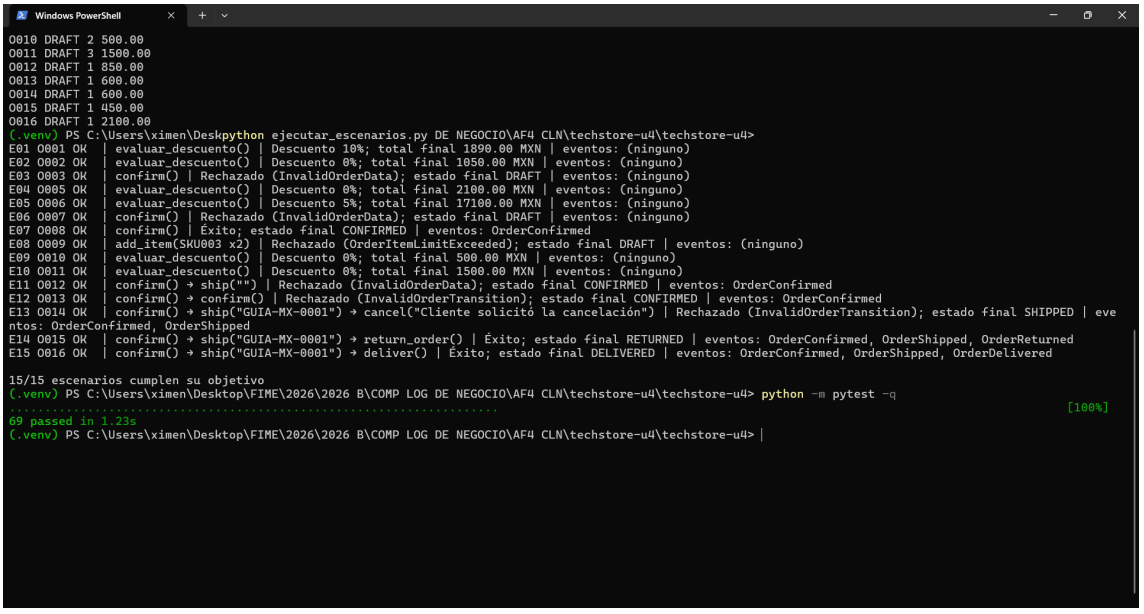


Reporte

Ejecutar escenarios:

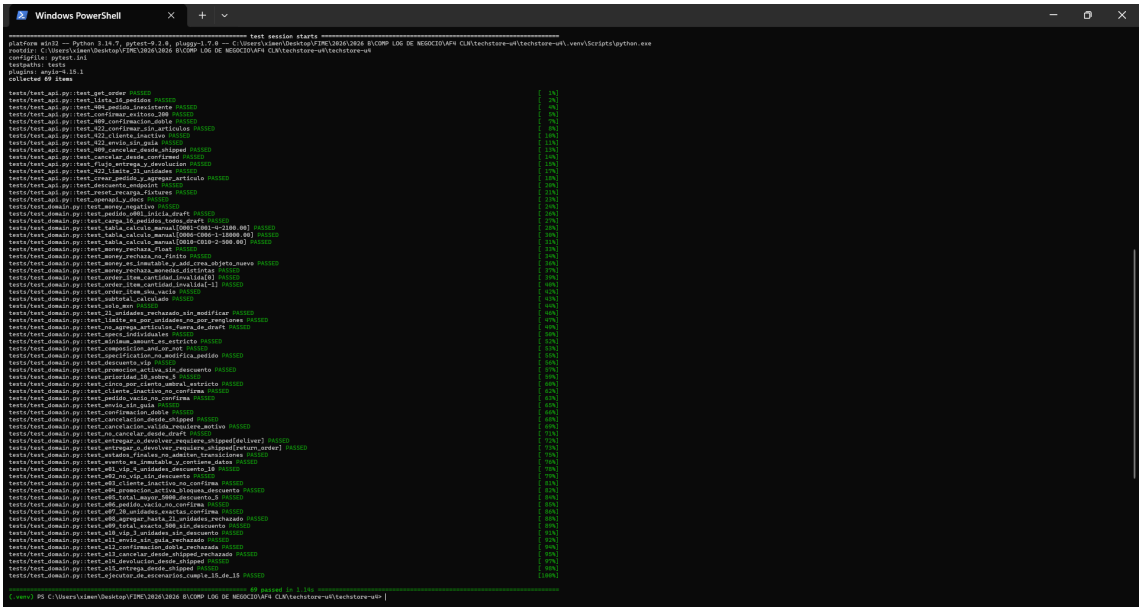


Pytest

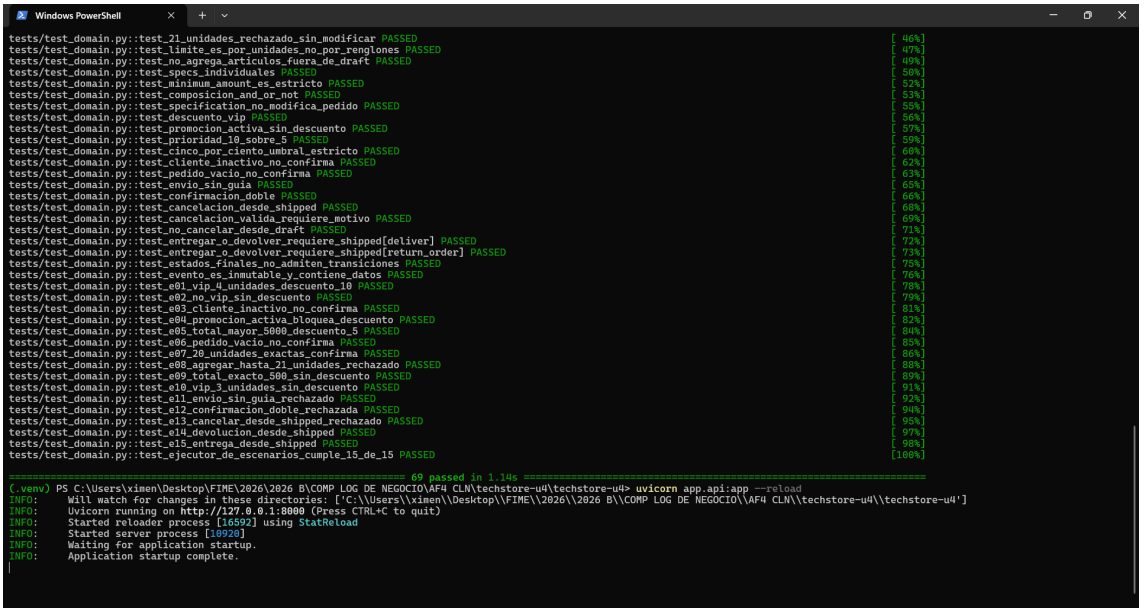


Reporte

Pytest con detalles



Application Startup



Reporte

API REST con FastAPI

TechStore U4 1.0.0 OAS3

/openapi-30.json

Administración de pedidos TechStore. Las reglas viven en el dominio (Order, Money, Specifications); la API solo las expone. Repositorio en memoria: reinicie el servidor o use POST /reset para volver a cargar los fixtures CSV.

Consultas

- GET `/orders` List Orders
- GET `/orders/{order_id}` Get Order
- GET `/orders/{order_id}/discount` Get Discount

Pedidos

- POST `/orders` Create Order
- POST `/orders/{order_id}/items` Add Item

Transiciones

- POST `/orders/{order_id}/confirm` Confirm Order
- POST `/orders/{order_id}/ship` Ship Order
- POST `/orders/{order_id}/cancel` Cancel Order
- POST `/orders/{order_id}/deliver` Deliver Order
- POST `/orders/{order_id}/return` Return Order

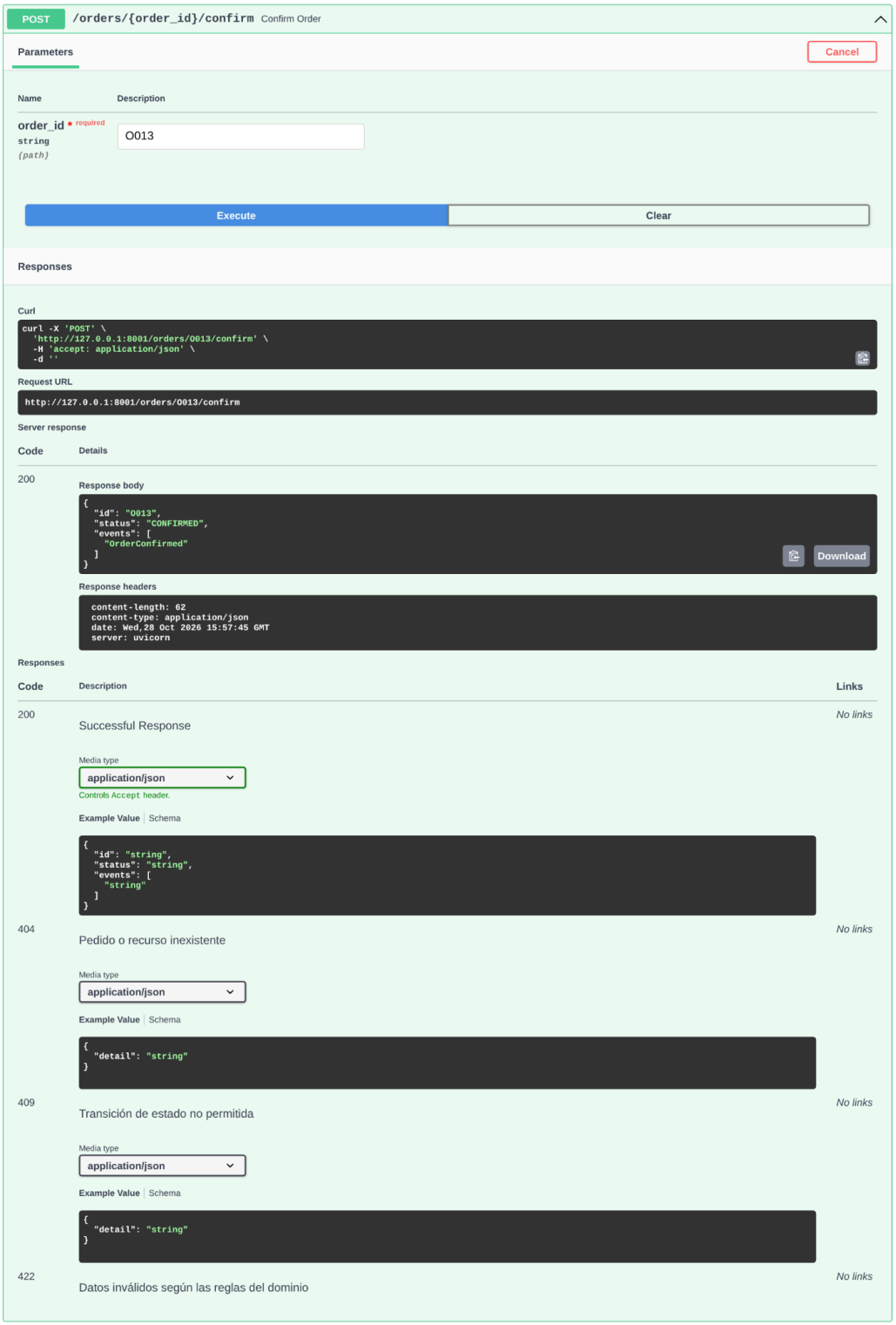
Laboratorio

- POST `/reset` Reset Fixtures

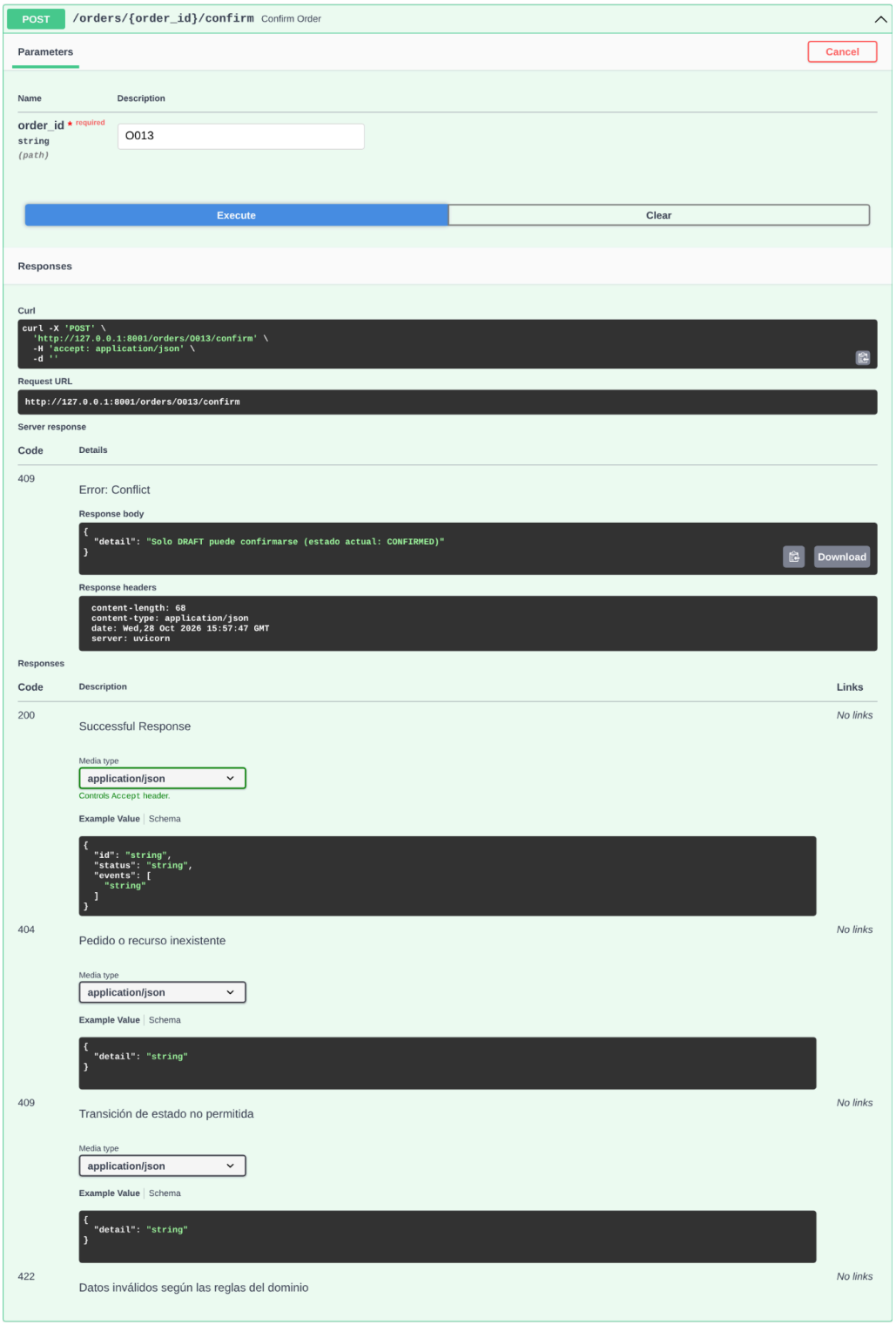
Schemas

- AddItemRequest
- CancelRequest
- CreateOrderRequest
- DiscountResponse
- ErrorResponse
- EventDTO
- HTTPValidationError
- ItemDTO
- OrderResponse
- ShipRequest
- StatusResponse
- ValidationError

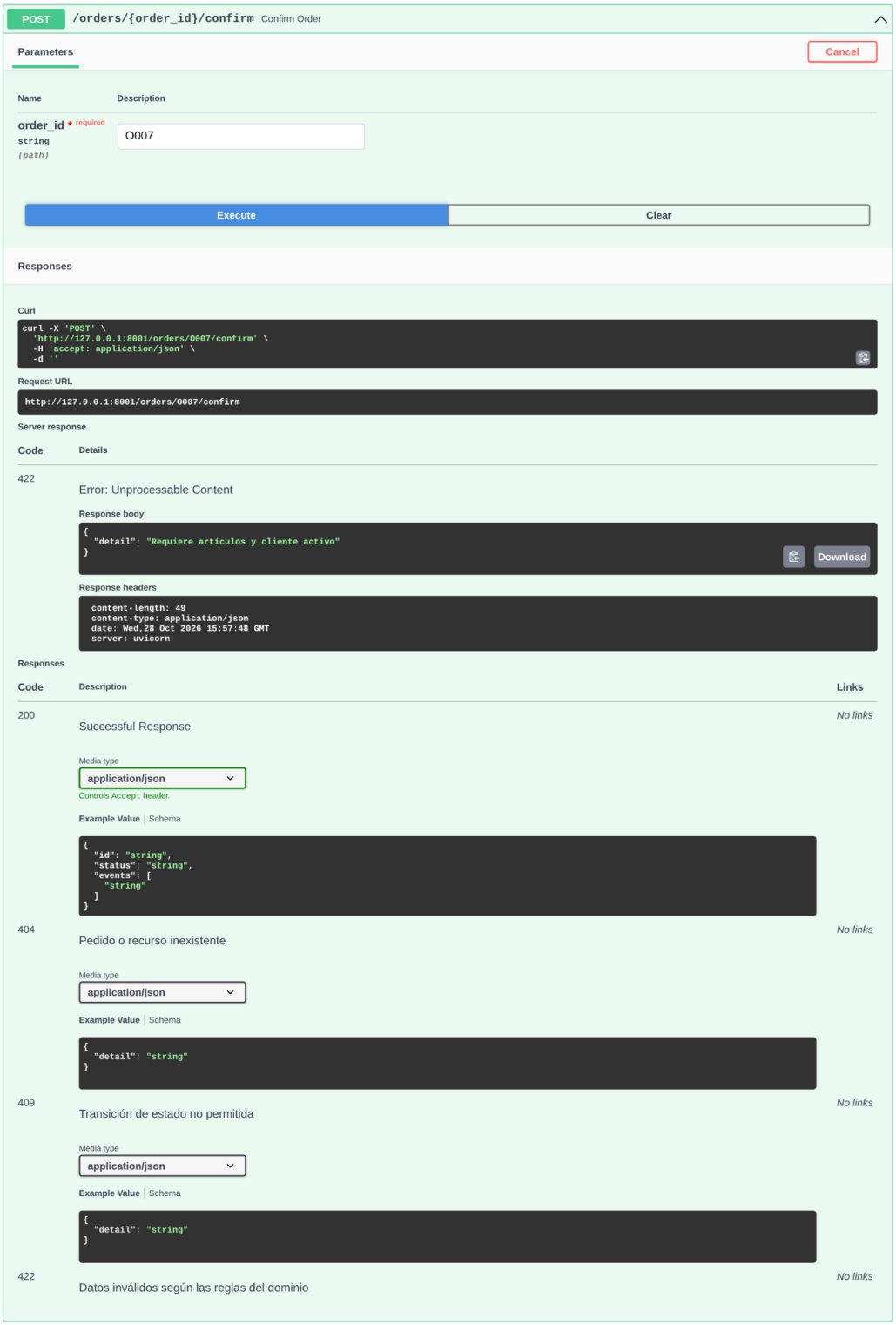
Reporte


POST /orders/{order_id}/confirm Confirm Order
Parameters
Cancel
Name
Description
order_id * required
string
(path)
O013
Execute
Clear
Responses
Curl
curl -X 'POST' \
'http://127.0.0.1:8001/orders/O013/confirm' \
-H 'accept: application/json' \
-d ''
Request URL
http://127.0.0.1:8001/orders/O013/confirm
Server response
Code
Details
200
Response body
{
"id": "O013",
"status": "CONFIRMED",
"events": [
"OrderConfirmed"
]
}
Download
Response headers
content-length: 62
content-type: application/json
date: Wed,28 Oct 2026 15:57:45 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
{
"id": "string",
"status": "string",
"events": [
"string"
]
}
404
Pedido o recurso inexistente
No links
Media type
application/json
Example Value | Schema
{
"detail": "string"
}
409
Transición de estado no permitida
No links
Media type
application/json
Example Value | Schema
{
"detail": "string"
}
422
Datos inválidos según las reglas del dominio
No links

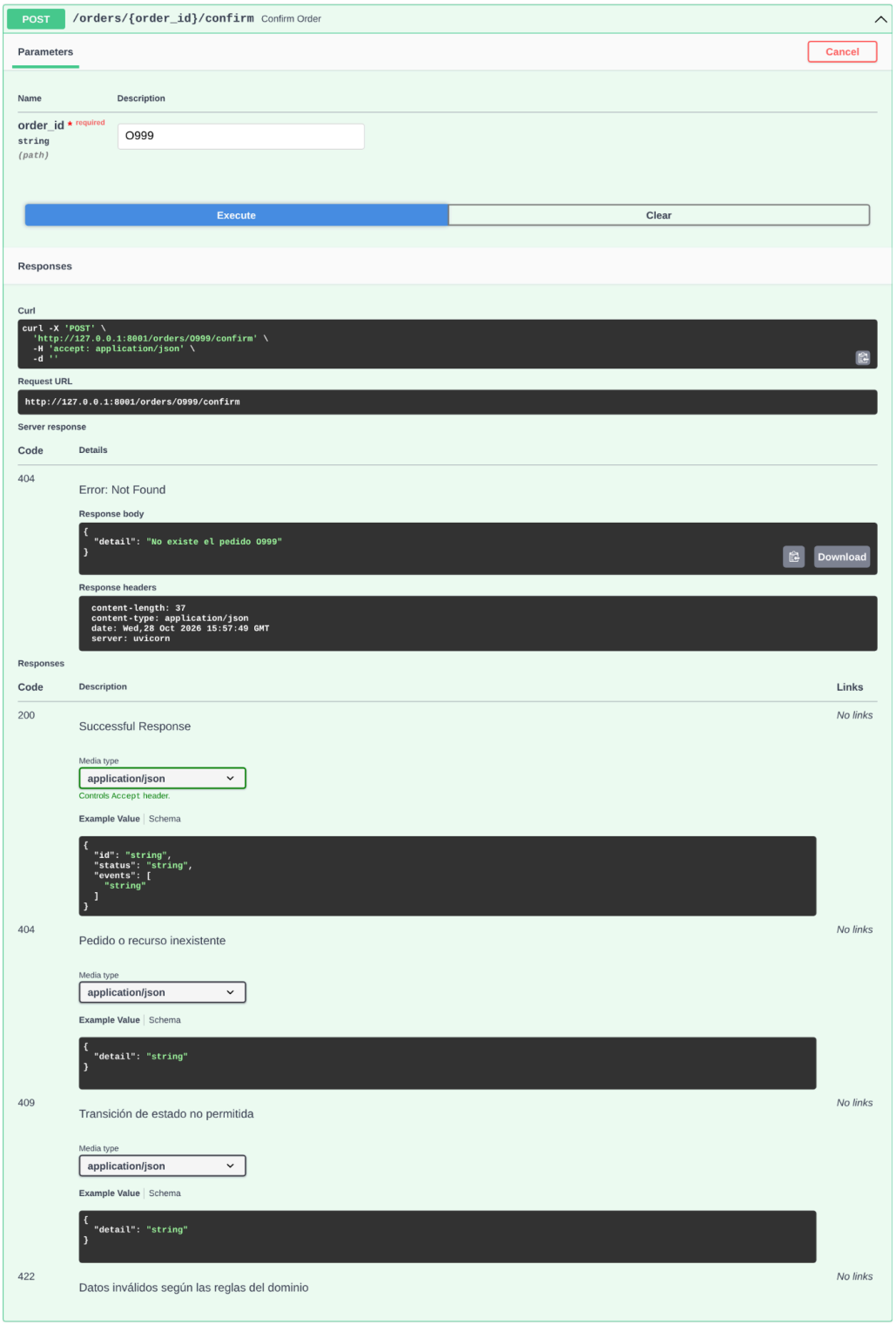
Reporte

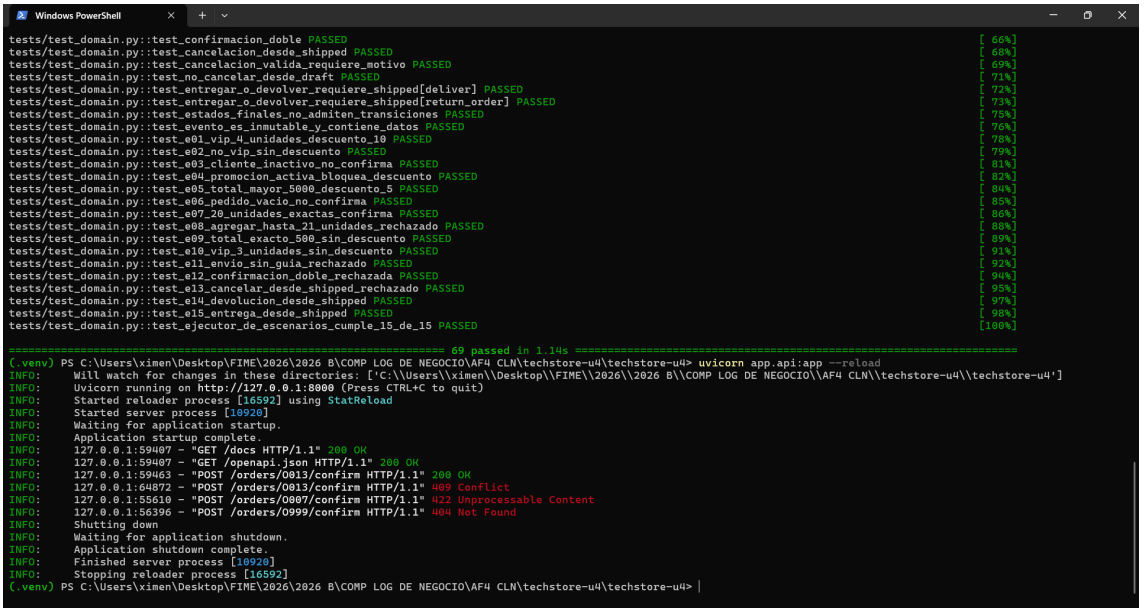

POST /orders/{order_id}/confirm Confirm Order
Parameters
Cancel
Name
Description
order_id * required
string
(path)
O013
Execute
Clear
Responses
Curl
curl -X 'POST' \
'http://127.0.0.1:8001/orders/O013/confirm' \
-H 'accept: application/json' \
-d ''
Request URL
http://127.0.0.1:8001/orders/O013/confirm
Server response
Code
Details
409
Error: Conflict
Response body
{
"detail": "Solo DRAFT puede confirmarse (estado actual: CONFIRMED)"
}
Download
Response headers
content-length: 68
content-type: application/json
date: Wed,28 Oct 2026 15:57:47 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
{
"id": "string",
"status": "string",
"events": [
"string"
]
}
404
Pedido o recurso inexistente
No links
Media type
application/json
Example Value | Schema
{
"detail": "string"
}
409
Transición de estado no permitida
No links
Media type
application/json
Example Value | Schema
{
"detail": "string"
}
422
Datos inválidos según las reglas del dominio
No links

Reporte


POST /orders/{order_id}/confirm Confirm Order
Parameters
Cancel
Name
Description
order_id * required
string
(path)
O007
Execute
Clear
Responses
Curl
curl -X 'POST' \
'http://127.0.0.1:8001/orders/O007/confirm' \
-H 'accept: application/json' \
-d ''
Request URL
http://127.0.0.1:8001/orders/O007/confirm
Server response
Code
Details
422
Error: Unprocessable Content
Response body
{
"detail": "Requiere articulos y cliente activo"
}
Download
Response headers
content-length: 49
content-type: application/json
date: Wed,28 Oct 2026 15:57:48 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
{
"id": "string",
"status": "string",
"events": [
"string"
]
}
404
Pedido o recurso inexistente
No links
Media type
application/json
Example Value | Schema
{
"detail": "string"
}
409
Transición de estado no permitida
No links
Media type
application/json
Example Value | Schema
{
"detail": "string"
}
422
Datos inválidos según las reglas del dominio
No links

Reporte


POST /orders/{order_id}/confirm Confirm Order
Parameters
Cancel
Name
Description
order_id * required
string
(path)
O999
Execute
Clear
Responses
Curl
curl -X 'POST' \
'http://127.0.0.1:8001/orders/O999/confirm' \
-H 'accept: application/json' \
-d ''
Request URL
http://127.0.0.1:8001/orders/O999/confirm
Server response
Code
Details
404
Error: Not Found
Response body
{
"detail": "No existe el pedido O999"
}
Download
Response headers
content-length: 37
content-type: application/json
date: Wed,28 Oct 2026 15:57:49 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
{
"id": "string",
"status": "string",
"events": [
"string"
]
}
404
Pedido o recurso inexistente
No links
Media type
application/json
Example Value | Schema
{
"detail": "string"
}
409
Transición de estado no permitida
No links
Media type
application/json
Example Value | Schema
{
"detail": "string"
}
422
Datos inválidos según las reglas del dominio
No links



Conclusiones:

La actividad permitió comprobar que colocar las reglas de negocio dentro del dominio, y no en la API, hace que el sistema sea más confiable y fácil de mantener. Usar Decimal evitó errores de redondeo en los importes, el Specification Pattern permitió combinar condiciones de descuento sin duplicar código, y registrar los eventos solo después de una transición válida garantizó que una operación rechazada no dejara rastros falsos. Las pruebas automáticas resultaron fundamentales: al romper reglas a propósito, fallaron de inmediato, lo que demostró que realmente protegen el comportamiento esperado. Lo aprendido se puede trasladar a otros sistemas, como inscripciones o reservas, donde también existen precondiciones, estados, eventos y pruebas.